# Software Engineering

## 20CS5PCSEG – Unit 4

Prof. SNEHA S BAGALKOT
Assistant Professor, Dept. of CSE

# UNIT – 4

- **Software Project Management**: The Management Spectrum, Product, process and project, The W5HH principle, Critical practices, Estimation for Software Project: Software Project estimation, Decomposition Techniques, Empirical Estimation models
- **Project Scheduling**: Basic Concepts, Project Scheduling, Defining Task set and Task network, Scheduling
- **Risk Management**: Reactive versus proactive strategies, Software Risks, Risk identification, Risk mitigation, monitoring and management, The RMMM plan

# Project Management Concepts

*Software Engineering: A Practitioner's Approach, 7/e*

**by Roger S. Pressman**

# The Four P's

- People — People must be organized to perform s/w work
- Product — Communicating with stake holders for requirements
- Process — Select appropriate process for people and product
- Project — Project must be planned by estimating effort and calendar time to accomplish work tasks like:

- Defining work
- Establishing checkpoints
- Identifying mechanisms for controlling and monitoring.

# Effective s/w Project Management focuses on the Four P's

- People — the most important element of a successful project
- Product — the software to be built
- Process — the set of framework activities and software engineering tasks to get the job done
- Project — all work required to make the product a reality

- Managers who forgets SE is an human endeavour – never have success in project mgmt.
- Manager who fails stakeholder communication – risks building solution irrelevant to problem
- Manager who pays little attention to process – risks of misusing technical methods and tools
- Manager who embarks without solid PM – jeopardizes the success of project

# The management spectrum

- People
- Goal is cultivation of motivated, highly skilled s/w people i.e. people factor is very important
- Thus SE institute developed a people capability Maturity model (People-CMM) that specifies that every organization needs to continually improve its ability to attract, develop, motivate, organize and retain the workforce to accomplish business objectives
- People –CMM is a companion to S/w-CMM in creating mature product

- Product

Before planning for project :

1. Product Objectives and scope should be established
2. Alternative solutions should be identified
3. Technical constraints and management constraints should be identified

- Objectives identify the overall goals for the product (from stakeholders point of view)
- Scope identifies the primary data, functions and behaviors that characterized the product

  - If these two things are not done: it is impossible to define estimates of cost, effective assessment of risk, a realistic breakdown of project tasks, or a manageable project schedule that provides indication of progress

- Process

❖ A software process provides the framework fro which a comprehensive plan for s/w development can be established

❖ A number of task sets: tasks, milestones, work products and quality assurance points enable framework activities to be adapted for project

❖ Finally umbrella activities such as: quality assurance, s/w configuration management and measurement overlay the process model

❖ These umbrella activities are independent of any framework activity and occur throughout the process

■ Project

❖ The primary reason for conducting planned and controlled s/w projects as it is the only known way to manage complexity and still s/w teams struggle.

❖ A study of 250 large s/w projects between 1998 and 2004 shows:

✔ 25 were deemed successful in that they achieved their schedule, cost and quality objectives

✔ 5o had delays or overruns below 35 percent

✔ 175 experienced major delays and overruns or were terminated without completion

To avoid project failure

❖ S/w project manager and s/w engineers who build the product must avoid a set of common warning signs, understand the critical success factors that lead to good project management and develop a commonsense approach for planning, monitoring and controlling the project

# People

- In a study three vice-presidents of major technology companies were asked what was the most important contributor to a successful s/w project

- VP1 : If one thing has to be picked then I'd say it is not the tools we use, it's the people
- VP2: most important ingredient for success of project was having smart people and the organizations ability to recruit good people
- VP3: only rule I have in management is to ensure I have good people…. Real good people and I grow good people and provide an environment in which good people can produce

- The next section discusses the stakeholders who participate in the software process and the manner in which they are organized to perform effective SE

# Stakeholders can be categorized into one of five constituencies

1. *Senior managers* who define the business issues that often have significant influence on the project.
2. *Project (technical) managers* who must plan, motivate, organize, and control the practitioners who do software work.
3. *Practitioners* who deliver the technical skills that are necessary to engineer a product or application.
4. *Customers* who specify the requirements for the software to be engineered and other stakeholders who have a peripheral interest in the outcome.
5. *End-users* who interact with the software once it is released for production use.

Every project is populated by people who fall within this taxonomy. To be effective the project team must be organized in a way that maximizes each persons skills and abilities – This is the job of team leader

# Software Teams

How to lead?

How to organize?

How to collaborate?

How to motivate?

How to create good ideas?

# Team Leader

- The MOI Model
  - **Motivation.** The ability to encourage (by “push or pull”) technical people to produce to their best ability.
  - **Organization.** The ability to mold existing processes (or invent new ones) that will enable the initial concept to be translated into a final product.
  - **Ideas or innovation.** The ability to encourage people to create and feel creative even when they must work within bounds established for a particular software product or application.

# Software Teams

***The following factors must be considered when selecting a software project team structure ...***

- the difficulty of the problem to be solved
- the size of the resultant program(s) in lines of code or function points
- the time that the team will stay together (team lifetime)
- the degree to which the problem can be modularized
- the required quality and reliability of the system to be built
- the rigidity of the delivery date
- the degree of sociability (communication) required for the project

# Organizational Paradigms

- closed paradigm—structures a team along a traditional hierarchy of authority
- random paradigm—structures a team loosely and depends on individual initiative of the team members
- open paradigm—attempts to structure a team in a manner that achieves some of the controls associated with the closed paradigm but also much of the innovation that occurs when using the random paradigm
- synchronous paradigm—relies on the natural compartmentalization of a problem and organizes team members to work on pieces of the problem with little active communication among themselves

*suggested by Constantine [Con93]*

# Avoid Team “Toxicity”

- A frenzied work atmosphere in which team members waste energy and lose focus on the objectives of the work to be performed.
- High frustration caused by personal, business, or technological factors that cause friction among team members.
- “Fragmented or poorly coordinated procedures” or a poorly defined or improperly chosen process model that becomes a roadblock to accomplishment.
- Unclear definition of roles resulting in a lack of accountability and resultant finger-pointing.
- “Continuous and repeated exposure to failure” that leads to a loss of confidence and a lowering of morale.

# Agile Teams

- Team members must have trust in one another.
- The distribution of skills must be appropriate to the problem.
- Mavericks may have to be excluded from the team, if team cohesiveness is to be maintained.
- Team is “self-organizing”
  - An adaptive team structure
  - Uses elements of Constantine’s random, open, and synchronous paradigms
  - Significant autonomy

# Team Coordination & Communication

- *Formal, impersonal approaches* include software engineering documents and work products (including source code), technical memos, project milestones, schedules, and project control tools, change requests and related documentation, error tracking reports, and repository data.
- *Formal, interpersonal procedures* focus on quality assurance activities applied to software engineering work products. These include status review meetings and design and code inspections.

# Team Coordination & Communication

- *Informal, interpersonal procedures* include group meetings for information dissemination and problem solving and "collocation of requirements and development staff."
- *Electronic communication* encompasses electronic mail, electronic bulletin boards, and by extension, video-based conferencing systems.
- *Interpersonal networking* includes informal discussions with team members and those outside the project who may have experience or insight that can assist team members.

# The Product Scope

- Scope
  - **Context.** How does the software to be built fit into a larger system, product, or business context and what constraints are imposed as a result of the context?
  - **Information objectives.** What customer-visible data objects (Chapter 8) are produced as output from the software? What data objects are required for input?
  - **Function and performance.** What function does the software perform to transform input data into output? Are any special performance characteristics to be addressed?
- Software project scope must be unambiguous and understandable at the management and technical levels.

# Problem Decomposition

- Sometimes called *partitioning* or *problem elaboration*
- Once scope is defined …
  - It is decomposed into constituent functions
  - It is decomposed into user-visible data objects

  *or*
  - It is decomposed into a set of problem classes
- Decomposition process continues until all functions or problem classes have been defined

# The Process

- Once a process framework has been established
  - Consider project characteristics
  - Determine the degree of rigor required
  - Define a task set for each software engineering activity
    - Task set =
      - Software engineering tasks
      - Work products
      - Quality assurance points
      - Milestones

# Melding the Problem and the Process

| COMMON PROCESS FRAMEWORK ACTIVITIES | communication | planning | modeling | construction | deployment |
|---|---|---|---|---|---|
| Software Engineering Tasks | | | | | |
| Product Functions | | | | | |
| Text input | | | | | |
| Editing and formatting | | | | | |
| Automatic copy edit | | | | | |
| Page layout capability | | | | | |
| Automatic indexing and TOC | | | | | |
| File management | | | | | |
| Document production | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

# The Project

- *Projects get into trouble when …*
  - Software people don’t understand their customer’s needs.
  - The product scope is poorly defined.
  - Changes are managed poorly.
  - The chosen technology changes.
  - Business needs change [or are ill-defined].
  - Deadlines are unrealistic.
  - Users are resistant.
  - Sponsorship is lost [or was never properly obtained].
  - The project team lacks people with appropriate skills.
  - Managers [and practitioners] avoid best practices and lessons learned.

# Common-Sense Approach to Projects

- *Start on the right foot.* This is accomplished by working hard (very hard) to understand the problem that is to be solved and then setting realistic objectives and expectations.
- *Maintain momentum. The* project manager must provide incentives to keep turnover of personnel to an absolute minimum, the team should emphasize quality in every task it performs, and senior management should do everything possible to stay out of the team's way.
- *Track progress.* For a software project, progress is tracked as work products (e.g., models, source code, sets of test cases) are produced and approved (using formal technical reviews) as part of a quality assurance activity.
- *Make smart decisions.* In essence, the decisions of the project manager and the software team should be to "keep it simple."
- *Conduct a postmortem analysis.* Establish a consistent mechanism for extracting lessons learned for each project.

# The W5HH principle: To Get to the Essence of a Project

- Why is the system being developed?
- What will be done?
- When will it be accomplished?
- Who is responsible?
- Where are they organizationally located?
- How will the job be done technically and managerially?
- How much of each resource (e.g., people, software, tools, database) will be needed?

***Barry Boehm [Boe96]***

# Critical Practices

- Formal risk management
- Empirical cost and schedule estimation
- Metrics-based project management
- Earned value tracking
- Defect tracking against quality targets
- People aware project management

# Estimation for Software Projects

*Software Engineering: A Practitioner's Approach, 7/e*
*Chapter 26*
**by Roger S. Pressman**

# Software Project Planning

The overall goal of project planning is to establish a pragmatic strategy for controlling, tracking, and monitoring a complex technical project.

Why?
*So the end result gets done on time, with quality!*

# Project Planning Task Set-I

- Establish project scope
- Determine feasibility
- Analyze risks
- Define required resources
  - Determine require human resources
  - Define reusable software resources
  - Identify environmental resources

# Project Planning Task Set-II

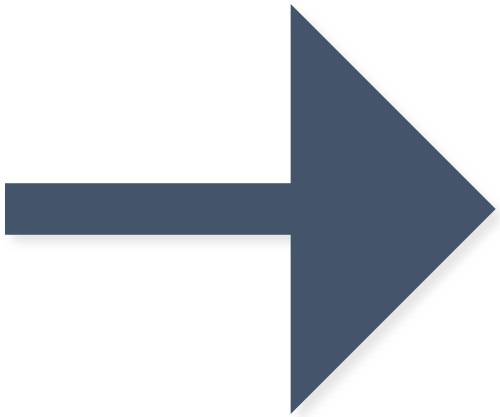
- Estimate cost and effort
  - Decompose the problem
  - Develop two or more estimates using size, function points, process tasks or use-cases
  - Reconcile the estimates
- Develop a project schedule
  - Establish a meaningful task set
  - Define a task network
  - Use scheduling tools to develop a timeline chart
  - Define schedule tracking mechanisms

# Estimation

- Estimation of resources, cost, and schedule for a software engineering effort requires
  - experience
  - access to good historical information (metrics)
  - the courage to commit to quantitative predictions when qualitative information is all that exists
- Estimation carries inherent risk and this risk leads to uncertainty

# Write it Down!

Project Scope
Estimates
Risks
Schedule
Control strategy



Software
Project
Plan

# To Understand Scope ...

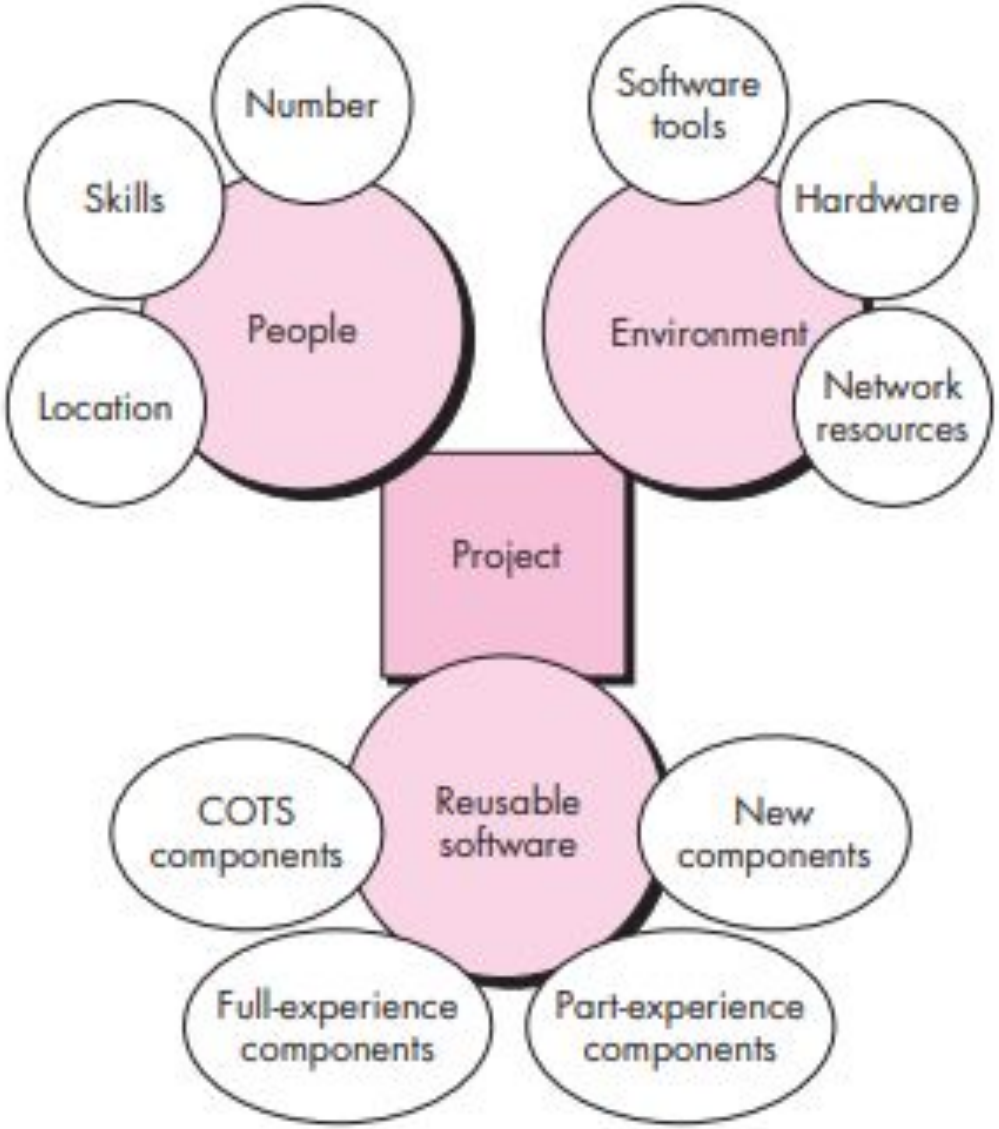
- Understand the customers needs
- understand the business context
- understand the project boundaries
- understand the customer's motivation
- understand the likely paths for change
- understand that ...

***Even when you understand,***
***nothing is guaranteed!***

# What is Scope?

- *Software scope* describes
  - the functions and features that are to be delivered to end-users
  - the data that are input and output
  - the "content" that is presented to users as a consequence of using the software
  - the performance, constraints, interfaces, and reliability that *bound* the system.
- Scope is defined using one of two techniques:
  - A narrative description of software scope is developed after communication with all stakeholders.
  - A set of use-cases is developed by end-users.

# Project Resources



# Project Estimation

- Project scope must be understood
- Elaboration (decomposition) is necessary
- Historical metrics are very helpful
- At least two different techniques should be used
- Uncertainty is inherent in the process

# Estimation Techniques

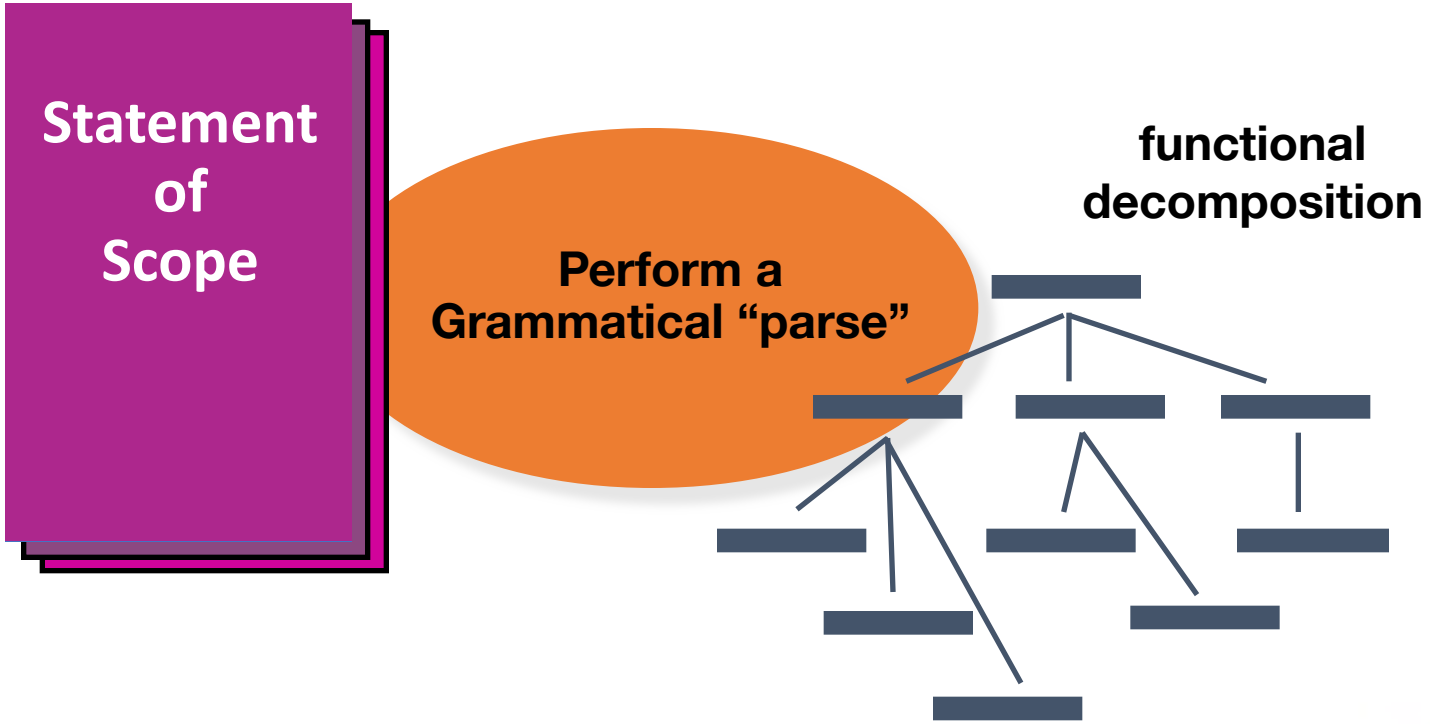
- Past (similar) project experience
- Conventional estimation techniques
  - task breakdown and effort estimates
  - size (e.g., FP) estimates
- Empirical models
- Automated tools

The "size" of software to be built can be estimated using a direct measure, LOC, or an indirect measure, FP.

# Estimation Accuracy

- Predicated on …
  - the degree to which the planner has properly estimated the size of the product to be built
  - the ability to translate the size estimate into human effort, calendar time, and dollars (a function of the availability of reliable software metrics from past projects)
  - the degree to which the project plan reflects the abilities of the software team
  - the stability of product requirements and the environment that supports the software engineering effort.

# Functional Decomposition



# Conventional Methods: LOC/FP Approach

- compute LOC/FP using estimates of information domain values
- use historical data to build estimates for the project

# Example: LOC Approach

| Function | Estimated LOC |
|---|---|
| User interface and control facilities (UICF) | 2,300 |
| Two-dimensional geometric analysis (2DGA) | 5,300 |
| Three-dimensional geometric analysis (3DGA) | 6,800 |
| Database management (DBM) | 3,350 |
| Computer graphics display facilities (CGDF) | 4,950 |
| Peripheral control function (PCF) | 2,100 |
| Design analysis modules (DAM) | 8,400 |
| *Estimated lines of code* | *33,200* |

Average productivity for systems of this type = 620 LOC/pm.

Burdened labor rate =$8000 per month, the cost per line of code is approximately $13.

Based on the LOC estimate and the historical productivity data, the total estimated project cost is **$431,000 and the estimated effort is 54 person-months.**

# Example: FP Approach

| Information domain value | Opt. | Likely | Pess. | Est. count | Weight | FP count |
|---|---|---|---|---|---|---|
| Number of external inputs | 20 | 24 | 30 | 24 | 4 | 97 |
| Number of external outputs | 12 | 15 | 22 | 16 | 5 | 78 |
| Number of external inquiries | 16 | 22 | 28 | 22 | 5 | 88 |
| Number of internal logical files | 4 | 4 | 5 | 4 | 10 | 42 |
| Number of external interface files | 2 | 2 | 3 | 2 | 7 | 15 |
| *Count total* | | | | | | *320* |

The estimated number of FP is derived:

$$FP_{estimated} = \text{count-total } 3\ [0.65 + 0.01\ 3\ S\ (F_i)]$$

$$FP_{estimated} = 375$$

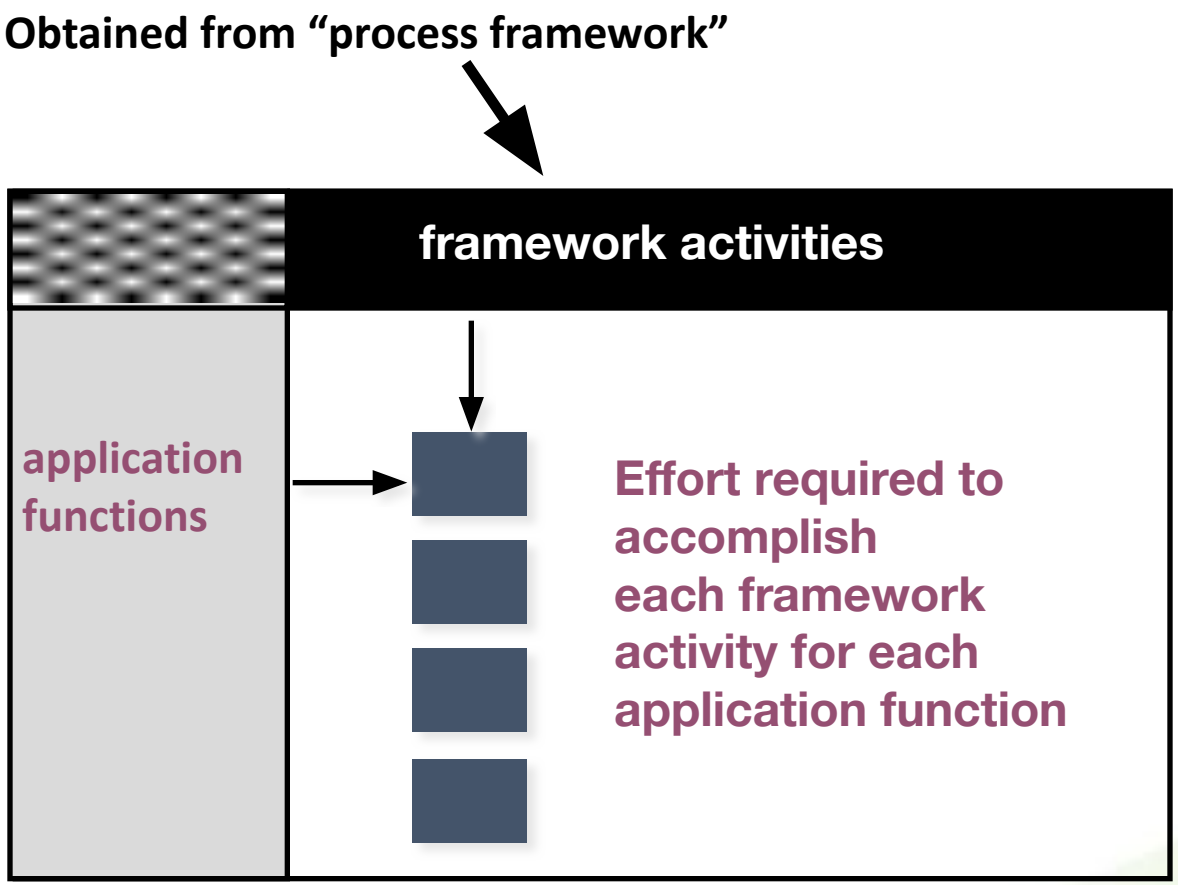
organizational average productivity = 6.5 FP/pm.

burdened labor rate = $8000 per month, approximately $1230/FP.

Based on the FP estimate and the historical productivity data, **total estimated project cost is $461,000 and estimated effort is 58 person-months.**

| Factor | Value |
|---|---|
| Backup and recovery | 4 |
| Data communications | 2 |
| Distributed processing | 0 |
| Performance critical | 4 |
| Existing operating environment | 3 |
| Online data entry | 4 |
| Input transaction over multiple screens | 5 |
| Master files updated online | 3 |
| Information domain values complex | 5 |
| Internal processing complex | 5 |
| Code designed for reuse | 4 |
| Conversion/installation in design | 3 |
| Multiple installations | 5 |
| Application designed for change | 5 |
| **Value adjustment factor** | **1.17** |

# Process-Based Estimation



# Process-Based Estimation Example

| Activity → | CC | Planning | Risk analysis | Engineering | | Construction release | | CE | Totals |
|---|---|---|---|---|---|---|---|---|---|
| Task → | | | | Analysis | Design | Code | Test | | |
| | | | | | | | | | |
| Function | | | | | | | | | |
| ↓ | | | | | | | | | |
| UICF | | | | 0.50 | 2.50 | 0.40 | 5.00 | n/a | 8.40 |
| 2DGA | | | | 0.75 | 4.00 | 0.60 | 2.00 | n/a | 7.35 |
| 3DGA | | | | 0.50 | 4.00 | 1.00 | 3.00 | n/a | 8.50 |
| CGDF | | | | 0.50 | 3.00 | 1.00 | 1.50 | n/a | 6.00 |
| DBM | | | | 0.50 | 3.00 | 0.75 | 1.50 | n/a | 5.75 |
| PCF | | | | 0.25 | 2.00 | 0.50 | 1.50 | n/a | 4.25 |
| DAM | | | | 0.50 | 2.00 | 0.50 | 2.00 | n/a | 5.00 |
| | | | | | | | | | |
| | | | | | | | | | |
| *Totals* | 0.25 | 0.25 | 0.25 | 3.50 | 20.50 | 4.50 | 16.50 | | 46.00 |
| | | | | | | | | | |
| *% effort* | 1% | 1% | 1% | 8% | 45% | 10% | 36% | | |

CC = customer communication CE = customer evaluation

Based on an average burdened labor rate of $8,000 per month, **the total estimated project cost is $368,000 and the estimated effort is 46 person-months.**

# Tool-Based Estimation

project characteristics →

calibration factors →

LOC/FP data →

# Estimation with Use-Cases

| | use cases | scenarios | pages | scenarios | pages | LOC | LOC estimate |
|---|---|---|---|---|---|---|---|
| User interface subsystem | 6 | 10 | 6 | 12 | 5 | 560 | 3,366 |
| Engineering subsystem group | 10 | 20 | 8 | 16 | 8 | 3100 | 31,233 |
| Infrastructure subsystem group | 5 | 6 | 5 | 10 | 6 | 1650 | 7,970 |
| | | | | | | | |
| Total LOC estimate | | | | | | | 42,568 |

Using 620 LOC/pm as the average productivity for systems of this type and a burdened labor rate of $8000 per month, the cost per line of code is approximately $13. Based on the use-case estimate and the historical productivity data, **the total estimated project cost is $552,000 and the estimated effort is 68 person-months.**

# Numerical Problems for Cost Estimation

## Methods for finding Cost Estimation

1. LOC Method
2. FP Method
3. Use-Case Method

# LOC Method Parameters

| Parameters | Measurement Unit |
| --- | --- |
| •Total LOC | x lines of code |
| •Cost per line of code | $x |
| •Labour rate per month | $x per month |
| •Organization's Average Productivity[OAP] | LOC/month |
| •Total Cost of the project | $x |
| •Estimated Effort in Person Month | x person per month |

# LOC Method Formulae

i) Total LOC : S = (Sopt + 4 * Sm + Spess ) / 6

ii) Cost per line of code = Labour rate per month / OAP

iii) Total Cost of the project = Total LOC * Cost per line of code

iv) Estimated Effort in Person Month = Total LOC / OAP

# LOC Method

**Question 1:**

An organization's average productivity[OAP] is 620 LOC/month. The average labour rate is $8000 per month. Calculate:

i)Total LOC using data in table

ii)Cost per line of code

iii)Total Cost of the project

iv)Estimated Effort in Person Month

| S/w Company | 3D Geo Analysis |
|---|---|
| Optimistic LoC | 4600 loc |
| Most Likely | 6900loc |
| Pessimistic | 8600loc |

# Solution for LOC Q1:

**Given:**

| S/w Company | 3D Geo Analysis |
|---|---|
| Optimistic LoC | 4600 loc |
| Most Likely | 6900loc |
| Pessimistic | 8600loc |

i) Total LOC : S = (Sopt + 4 * Sm + Spess ) / 6

= ( 4600 + 4* 6900 + 8600) / 6

= 33200 loc

# Contd..

**Given**: OAP = 620 LOC/month ; Average labour rate = $8000 per month

ii) Cost per line of code = Labour rate per month / OAP

= $8000 / 620 = $13

iii) Total Cost of the project = Total LOC * Cost per line of code

= 33200 * 13 = $431600

iv) Estimated Effort in Person Month = Total LOC / OAP

= 33200 / 620

= 54 person per month

# LOC Method

**Question 2:**

An organization's average productivity[OAP] is 2000 LOC/month. The average labour rate is $6000 per month and Total LOC is 60000 loc. Calculate:

i) Cost per line of code

ii) Total Cost of the project

iii) Estimated Effort in Person Month

**Answer:**

**Cost per loc=$3**

**Find total cost? $1,80,000**

**Effort in person month? 30**

# LOC Method

**Question 3:**

If a team produces 800 line of code per month at the burdened labour rate of $8000 per month. Calculate the project cost and estimated in person-month for developing an information system which spans 32000 line of code?

**Answer:**

**Cost per loc=$10**

**Find total cost? $320000**

**Effort in person month? 40 person per month**

# LOC Method

(ii) An organizations average productivity is 650 LOC/pm. The average labor rate is $9750 per month. Calculate

a) Cost per line of Code
b) Overall Project cost if the proposed project has 128000 LOC
c) Estimated effort in person-months

An organization's average productivity is 550 LOC/pm. The average labor rate is $9750 per month. Calculate (i) Cost per line of Code (ii) Overall Project cost if the proposed project has 282000 LOC (iii) Estimated effort in person-months

# FP Method Parameters

| Parameters | Measurement Unit |
| --- | --- |
| •Value Adjustment Factor [VAF] | x [float] |
| •Total FP OR FP Estimate | x function points |
| •Cost per Function point | $x |
| •Labour rate per month | $x per month |
| •Organization's Average Productivity[OAP] | FP/month |
| •Total Cost of the project | $x |
| •Estimated Effort in Person Month | x person per month |

# FP Method Formulae

i) Value Adjustment Factor [VAF] = 0.65 + ( 0.01 * £(Fi) )

ii) FP estimate = FP * [0.65 + ( 0.01 * £(Fi) ) ]

i.e., FP estimate = FP * VAF [Also called 'Total FP']

iii) Cost per Function Point = Labour rate per month / OAP

iv) Total Cost of the project = Total FP * Cost per FP

v) Estimated Effort in Person Month = Total FP / OAP

# FP Method

**Question 1:**

An organization's average productivity[OAP] is 6.5 FP/month. The average labour rate is $8000 per month, FP is 320 and £(Fi) is 52. Calculate:

i) Value Adjustment Factor [VAF]

ii) FP estimate OR Total FP

iii) Cost per Function point

iv) Total Cost of the project

v) Estimated Effort in Person Month

# Solution for FP Q1:

**Given:** £(Fi) = 52 , FP = 320

i) Value Adjustment Factor [VAF] = 0.65 + ( 0.01 * £(Fi) )

= ( 0.65 + (0.01 * 52 )

= 1.17

ii) FP estimate = FP * VAF //If VAF is given, directly substitute here

= 320 * 1.17

= 375 FP

# Contd..

**Given**: OAP = 6.5 FP/month ; Average labour rate = $8000 per month

iii) Cost per Function Point = Labour rate per month / OAP

= $8000 / 6.5 = $1231

iv) Total Cost of the project = Total FP * Cost per FP

= 375 * 1231 = $461625

v) Estimated Effort in Person Month = Total FP / OAP

= 375 / 6.5

= 58 person per month

# FP Method

**Question 2:**

If an organization average productivity is 6 FP per month at the burdened labour rate of $6000 per month. Given the count total of 320 (Information Domain Value count total) and £(Fi) is 52 calculate

i. FP estimate //Note: FP estimate = FP * [0.65 + ( 0.01 * £(Fi) ) ]

ii. Cost per FP

iii. Total Project cost

iv. Estimated effort in person month

# FP Method

**Question 3:**

XYZ company has a project contract with FP estimated to be 360. If the organization average productivity is 8 FP per month at the burdened labour rate of $8000 per month. Estimate the cost of the project and effort in person month. (No VAF and £(Fi) is given )

# FP Method

(i) An organizations average productivity is 6FP/pm. The average labor rate is $5600 per month. If a proposed project has a count total of 580 FP's and the Value Adjustment factor (VAF) is 40. Calculate:

a) Cost per Functional Point
b) Overall Project Cost
c) Estimated effort in person-months

An Organizations average productivity is 16FP/pm. The Average labor rate is $5600 per month. if the proposed project has a count total of 479 and VAF is 56. Calculate

i)Cost per Functional point
ii)Overall project cost
iii)Estimated effort in person months

An organization's average productivity is 12 FP/pm. The average labor rate is $15600 per month. If a proposed project has a count total of 560 and the Value Adjustment factor(VAF) is 34. Calculate: (i) Cost per Functional Point (ii) Overall Project Cost (iii) Estimated effort in person-months

# Use-Case Method Parameters

| Parameters | Measurement Unit |
| --- | --- |
| •Total LOC | x lines of code |
| •Cost per line of code | $x |
| •Labour rate per month | $x per month |
| •Organization's Average Productivity[OAP] | LOC/month |
| •Total Cost of the project | $x |
| •Estimated Effort in Person Month | x person per month |

# Use-Case Method Formulae

i) Cost per line of code = Labour rate per month / OAP

iii) Total Cost of the project = Total LOC * Cost per line of code

iv) Estimated Effort in Person Month = Total LOC / OAP

# Use-Case Method

**[This is Similar to LOC method]**

**Question:**

A company uses USE-CASE METHOD based cost estimation, They estimate that the system will require 42500 LOC. If the organization productivity rate is 620 LOC per month at the burdened labour rate of $8000 per month . Estimate the cost of project and effort in person month.

# Solution for Use-Case Q1:

**Given:** Total LOC = 42500 loc , OAP = 620 LOC pm , Labour rate = $8000 pm

i) Cost per line of code = Labour rate per month / OAP

= $8000 / 620 = $13

ii) Total Cost of the project = Total LOC * Cost per line of code

= 42500 * 13 = $552500

iii) Estimated Effort in Person Month = Total LOC / OAP

= 42500 / 620

= 69 person per month

# Empirical Estimation Models

***General form:***

effort = tuning coefficient * size$^{\text{exponent}}$

usually derived as person-months of effort required

either a constant or a number derived based on complexity of project

usually LOC but may also be function point

empirically derived

# COCOMO-II

- COCOMO II is actually a hierarchy of estimation models that address the following areas:
  - *Application composition model.* Used during the early stages of software engineering, when prototyping of user interfaces, consideration of software and system interaction, assessment of performance, and evaluation of technology maturity are paramount.
  - *Early design stage model.* Used once requirements have been stabilized and basic software architecture has been established.
  - *Post-architecture-stage model.* Used during the construction of the software.

# The Software Equation

*A dynamic multivariable model*

$$E = \frac{\text{LOC} \times B^{0.333}}{P^3} \times \frac{1}{t^4}$$

where

E = effort in person-months or person-years

t = project duration in months or years

B = “special skills factor”

P = “productivity parameter”

# Estimation for OO Projects-I

- Develop estimates using effort decomposition, FP analysis, and any other method that is applicable for conventional applications.
- Using object-oriented requirements modeling, develop use-cases and determine a count.
- From the analysis model, determine the number of key classes (called analysis classes).
- Categorize the type of interface for the application and develop a multiplier for support classes:
  - **Interface type** **Multiplier**
  - No GUI 2.0
  - Text-based user interface 2.25
  - GUI 2.5
  - Complex GUI 3.0

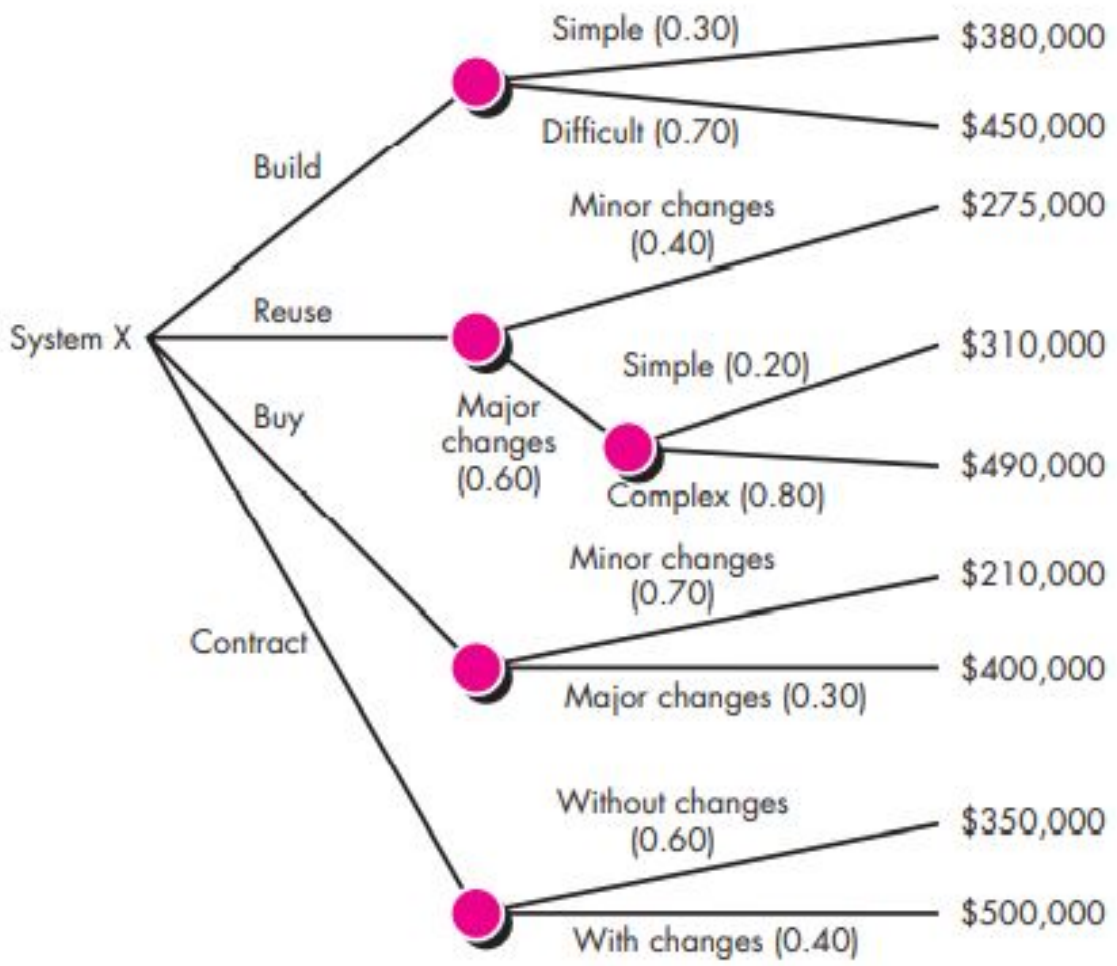
# Estimation for OO Projects-II

- Multiply the number of key classes (step 3) by the multiplier to obtain an estimate for the number of support classes.
- Multiply the total number of classes (key + support) by the average number of work-units per class. Lorenz and Kidd suggest 15 to 20 person-days per class.
- Cross check the class-based estimate by multiplying the average number of work-units per use-case

# Estimation for Agile Projects

- Each user scenario (a mini-use-case) is considered separately for estimation purposes.
- The scenario is decomposed into the set of software engineering tasks that will be required to develop it.
- Each task is estimated separately. Note: estimation can be based on historical data, an empirical model, or "experience."
  - Alternatively, the 'volume' of the scenario can be estimated in LOC, FP or some other volume-oriented measure (e.g., use-case count).
- Estimates for each task are summed to create an estimate for the scenario.
  - Alternatively, the volume estimate for the scenario is translated into effort using historical data.
- The effort estimates for all scenarios that are to be implemented for a given software increment are summed to develop the effort estimate for the increment.

# Decision tree to support the make/buy decision



# Computing Expected Cost

**Expected cost =**

$$\sum (\text{path probability})_i \times (\text{estimated path cost})_i$$

***For example, the expected cost to build is:***

$$\text{expected cost}_{build} = 0.30\ (\$380K) + 0.70\ (\$450K)$$
$$= \$429\ K$$

***similarly,***

$$\text{expected cost}_{reuse} = \$382K$$

$$\text{expected cost}_{buy} = \$267K$$

$$\text{expected cost}_{contr} = \$410K$$

# Project Scheduling

Chapter 27

*Software Engineering: A Practitioner's Approach, 7/e*

**by Roger S. Pressman**

# Why Are Projects Late?

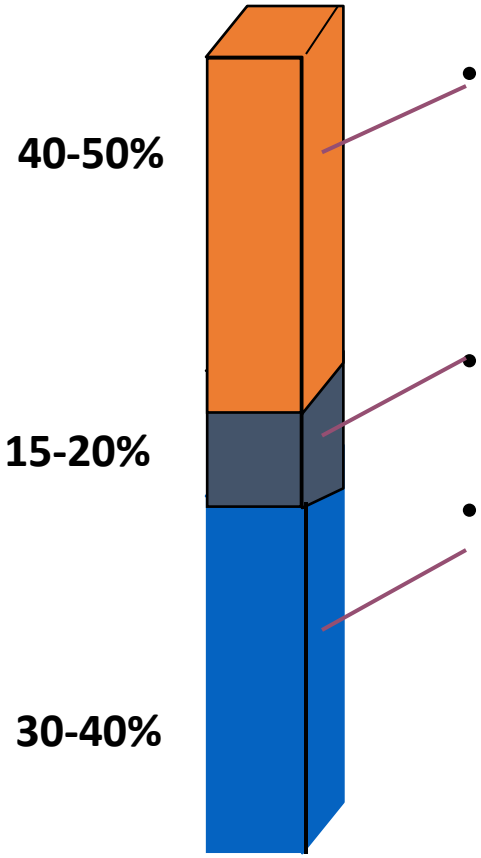
- an unrealistic deadline established by someone outside the software development group
- changing customer requirements that are not reflected in schedule changes;
- an honest underestimate of the amount of effort and/or the number of resources that will be required to do the job;
- predictable and/or unpredictable risks that were not considered when the project commenced;
- technical difficulties that could not have been foreseen in advance;
- human difficulties that could not have been foreseen in advance;
- miscommunication among project staff that results in delays;
- a failure by project management to recognize that the project is falling behind schedule and a lack of action to correct the problem

# Scheduling Principles

- compartmentalization—define distinct tasks
- interdependency—indicate task interrelationship
- effort validation—be sure resources are available
- defined responsibilities—people must be assigned
- defined outcomes—each task must have an output
- defined milestones—review for quality

# Effort and Delivery Time

# Effort Allocation




- “front end” activities
  - customer communication
  - analysis
  - design
  - review and modification
- construction activities
  - coding or code generation
- testing and installation
  - unit, integration
  - white-box, black box
  - regression

# Defining Task Sets

- determine type of project
- assess the degree of rigor required
- identify adaptation criteria
- select appropriate software engineering tasks

# Task Set Refinement

**1.1** Concept scoping determines the overall scope of the project.

is refined to

Task definition: Task 1.1 Concept Scoping

1.1.1 Identify need, benefits and potential customers;

1.1.2 Define desired output/control and input events that drive the application;

Begin Task 1.1.2

1.1.2.1 FTR: Review written description of need

FTR indicates that a formal technical review (Chapter 26) is to be conducted.

1.1.2.2 Derive a list of customer visible outputs/inputs

1.1.2.3 FTR: Review outputs/inputs with customer and revise as required;

endtask Task 1.1.2

1.1.3 Define the functionality/behavior for each major function;

Begin Task 1.1.3

1.1.3.1 FTR: Review output and input data objects derived in task 1.1.2;

1.1.3.2 Derive a model of functions/behaviors;

1.1.3.3 FTR: Review functions/behaviors with customer and revise as required;

endtask Task 1.1.3

1.1.4 Isolate those elements of the technology to be implemented in software;

1.1.5 Research availability of existing software;

1.1.6 Define technical feasibility;

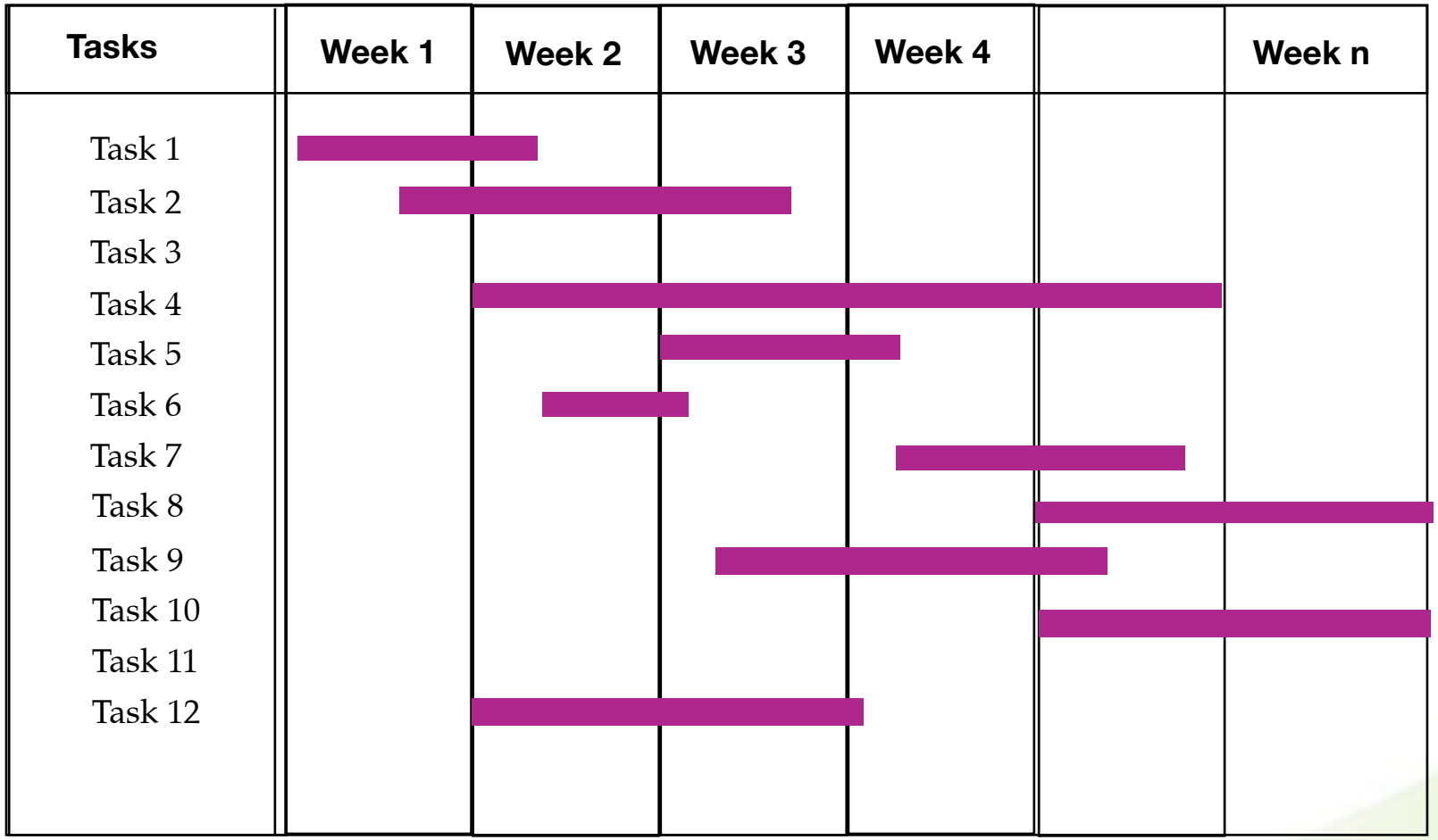
1.1.7 Make quick estimate of size;

1.1.8 Create a Scope Definition;

endTask definition: Task 1.1

# Define a Task Network

# Timeline Charts

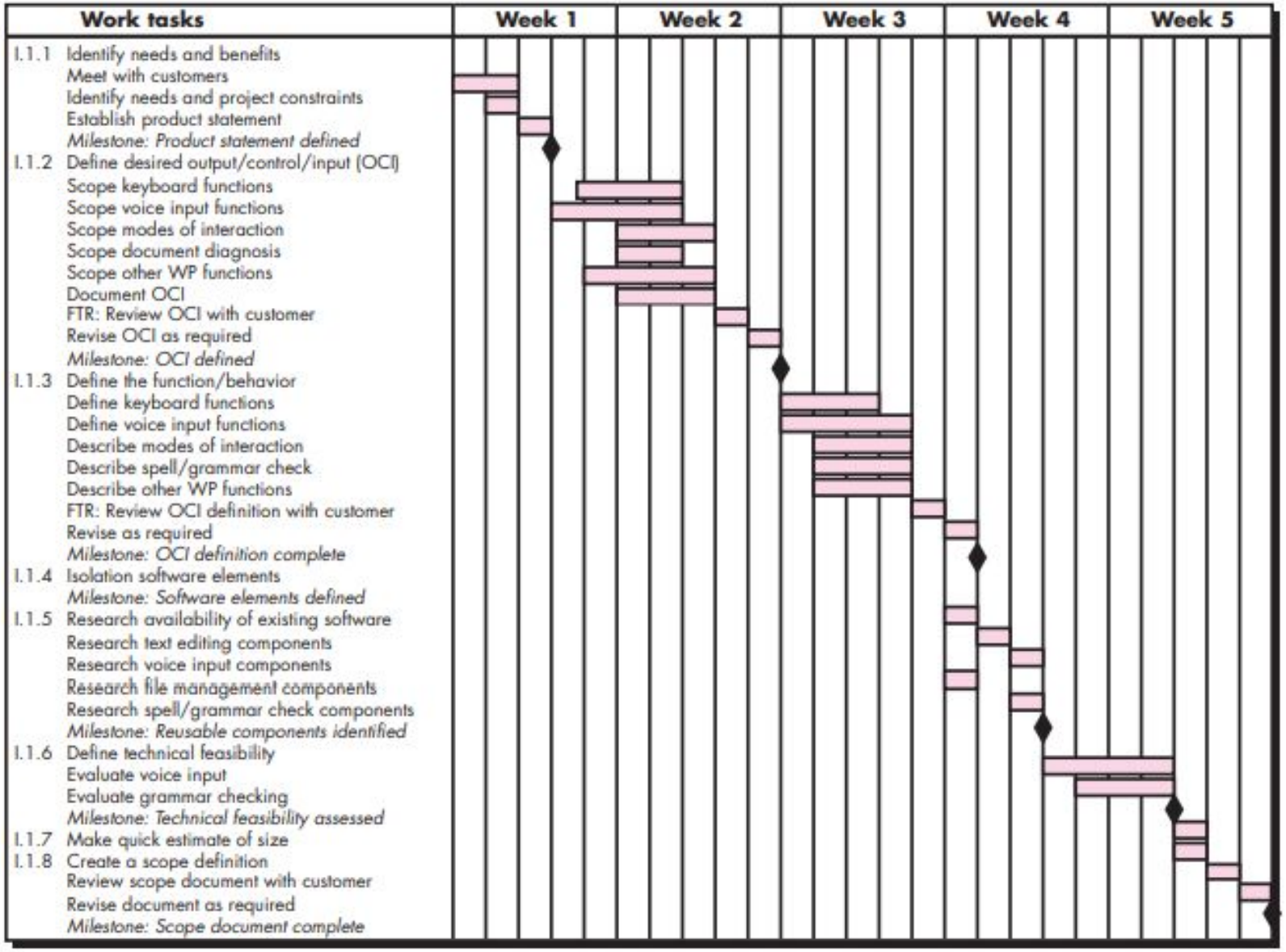


# Use Automated Tools to Derive a Timeline Chart

# Schedule Tracking

- conduct periodic project status meetings in which each team member reports progress and problems.
- evaluate the results of all reviews conducted throughout the software engineering process.
- determine whether formal project milestones (the diamonds shown in diagram ) have been accomplished by the scheduled date.
- compare actual start-date to planned start-date for each project task listed in the resource table.
- meet informally with practitioners to obtain their subjective assessment of progress to date and problems on the horizon.
- use earned value analysis to assess progress quantitatively.

# An example time-line chart



# An example project table

| Work tasks | Planned start | Actual start | Planned complete | Actual complete | Assigned person | Effort allocated | Notes |
|---|---|---|---|---|---|---|---|
| I.1.1 Identify needs and benefits | | | | | | | Scoping will require more effort/time |
| Meet with customers | wk1, d1 | wk1, d1 | wk1, d2 | wk1, d2 | BLS | 2 p-d | |
| Identify needs and project constraints | wk1, d2 | wk1, d2 | wk1, d2 | wk1, d2 | JPP | 1 p-d | |
| Establish product statement | wk1, d3 | wk1, d3 | wk1, d3 | wk1, d3 | BLS/JPP | 1 p-d | |
| *Milestone: Product statement defined* | wk1, d3 | wk1, d3 | wk1, d3 | wk1, d3 | | | |
| I.1.2 Define desired output/control/input (OCI) | | | | | | | |
| Scope keyboard functions | wk1, d4 | wk1, d4 | wk2, d2 | | BLS | 1.5 p-d | |
| Scope voice input functions | wk1, d3 | wk1, d3 | wk2, d2 | | JPP | 2 p-d | |
| Scope modes of interaction | wk2, d1 | | wk2, d3 | | MLL | 1 p-d | |
| Scope document diagnostics | wk2, d1 | | wk2, d2 | | BLS | 1.5 p-d | |
| Scope other WP functions | wk1, d4 | wk1, d4 | wk2, d3 | | JPP | 2 p-d | |
| Document OCI | wk2, d1 | | wk2, d3 | | MLL | 3 p-d | |
| FTR: Review OCI with customer | wk2, d3 | | wk2, d3 | | all | 3 p-d | |
| Revise OCI as required | wk2, d4 | | wk2, d4 | | all | 3 p-d | |
| *Milestone: OCI defined* | wk2, d5 | | wk2, d5 | | | | |
| I.1.3 Define the function/behavior | | | | | | | |

# Progress on an OO Project-I

- *Technical milestone: OO analysis completed*
  - All classes and the class hierarchy have been defined and reviewed.
  - Class attributes and operations associated with a class have been defined and reviewed.
  - Class relationships have been established and reviewed.
  - A behavioral model has been created and reviewed.
  - Reusable classes have been noted.

# Progress on an OO Project-I

- *Technical milestone: OO design completed*
  - The set of subsystems has been defined and reviewed.
  - Classes are allocated to subsystems and reviewed.
  - Task allocation has been established and reviewed.
  - Responsibilities and collaborations have been identified.
  - Attributes and operations have been designed and reviewed.
  - The communication model has been created and reviewed.

# Progress on an OO Project-II

- *Technical milestone: OO programming completed*
  - Each new class has been implemented in code from the design model.
  - Extracted classes (from a reuse library) have been implemented.
  - Prototype or increment has been built.

# Progress on an OO Project-II

- *Technical milestone: OO testing*
  - The correctness and completeness of OO analysis and design models has been reviewed.
  - A class-responsibility-collaboration network (Chapter 6) has been developed and reviewed.
  - Test cases are designed and class-level tests (Chapter 19) have been conducted for each class.
  - Test cases are designed and cluster testing (Chapter 19) is completed and the classes are integrated.
  - System level tests have been completed.

# Earned Value Analysis (EVA)

- Earned value
  - is a measure of progress
  - enables us to assess the “percent of completeness” of a project using quantitative analysis rather than rely on a gut feeling
  - “provides accurate and reliable readings of performance from as early as 15 percent into the project.” [Fle98]

# Computing Earned Value-I

- The *budgeted cost of work scheduled* (BCWS) is determined for each work task represented in the schedule.
  - $BCWS_i$ is the effort planned for work task *i.*
  - To determine progress at a given point along the project schedule, the value of BCWS is the sum of the $BCWS_i$ values for all work tasks that should have been completed by that point in time on the project schedule.
- The BCWS values for all work tasks are summed to derive the *budget at completion,* BAC. Hence,

$$BAC = \sum (BCWS_k) \text{ for all tasks } k$$

# Computing Earned Value-II

- Next, the value for *budgeted cost of work performed* (BCWP) is computed.
  - The value for BCWP is the sum of the BCWS values for all work tasks that have actually been completed by a point in time on the project schedule.
- "the distinction between the BCWS and the BCWP is that the former represents the budget of the activities that were planned to be completed and the latter represents the budget of the activities that actually were completed." [Wil99]
- Given values for BCWS, BAC, and BCWP, important progress indicators can be computed:
  - Schedule performance index, SPI = BCWP/BCWS
  - Schedule variance, SV = BCWP – BCWS
  - SPI is an indication of the efficiency with which the project is utilizing scheduled resources.

# Computing Earned Value-III

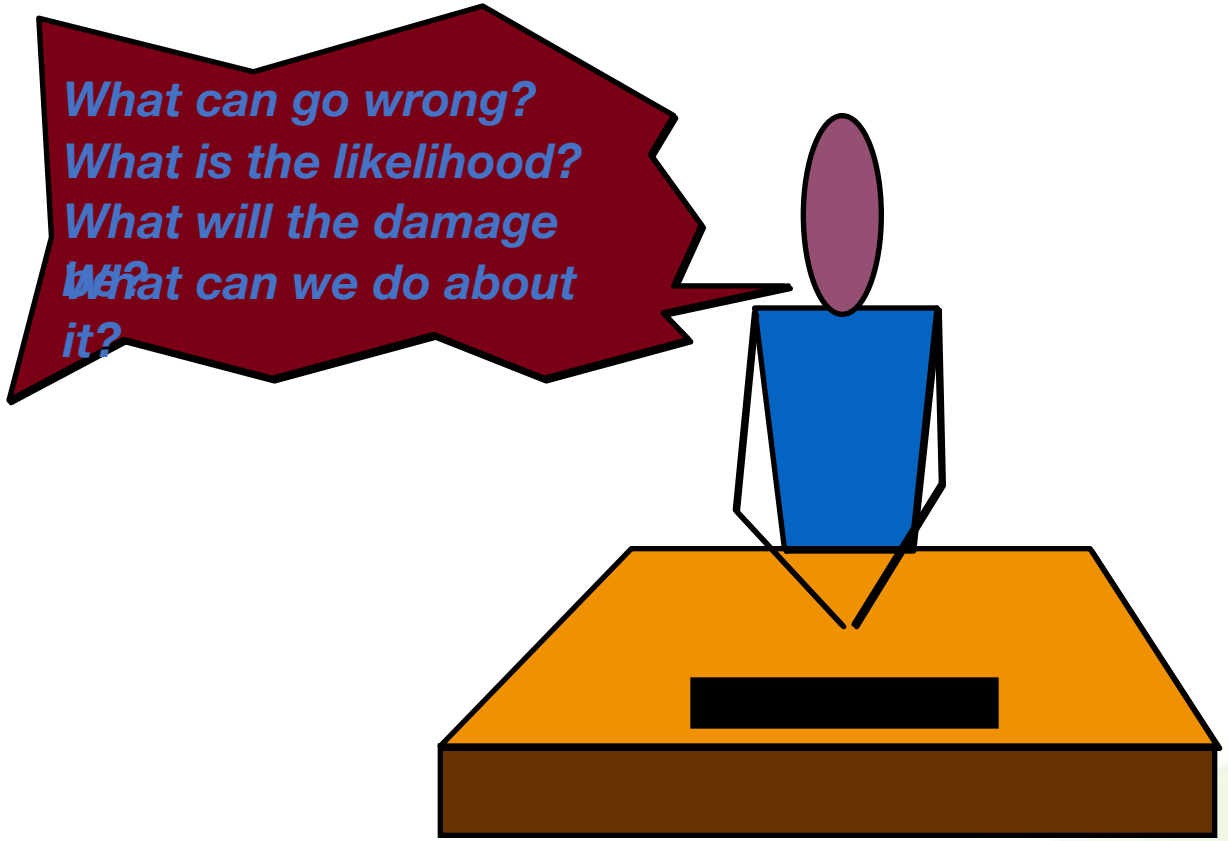
- Percent scheduled for completion = BCWS/BAC
  - provides an indication of the percentage of work that should have been completed by time *t.*
- Percent complete = BCWP/BAC
  - provides a quantitative indication of the percent of completeness of the project at a given point in time, *t.*
- *Actual cost of work performed,* ACWP, is the sum of the effort actually expended on work tasks that have been completed by a point in time on the project schedule. It is then possible to compute
  - Cost performance index, CPI = BCWP/ACWP
  - Cost variance, CV = BCWP – ACWP

# Risk Analysis

*Software Engineering: A Practitioner's Approach, 7/e, Chapter 28*

**by Roger S. Pressman**

# Project Risks



# Reactive Risk Management

- Project team reacts to risks when they occur
- Mitigation—plan for additional resources in anticipation of fire fighting
- Fix on failure—resource are found and applied when the risk strikes
- Crisis management—failure does not respond to applied resources and project is in jeopardy

# Proactive Risk Management

- Formal risk analysis is performed
- Organization corrects the root causes of risk
  - TQM concepts and statistical SQA
  - examining risk sources that lie beyond the bounds of the software
  - developing the skill to manage change

# Seven Principles

- **Maintain a global perspective**—view software risks within the context of system and the business problem
- **Take a forward-looking view**—think about the risks that may arise in the future;  establish contingency plans
- **Encourage open communication**—if someone states a potential risk, don't discount it.
- **Integrate**—a consideration of risk must be integrated into the software process
- **Emphasize a continuous process**—the team must be vigilant throughout the software process, modifying identified risks as more information is known and adding new ones as better insight is achieved.
- **Develop a shared product vision**—if all stakeholders share the same vision of the software, it likely that better risk identification and assessment will occur.
- **Encourage teamwork**—the talents, skills and knowledge of all stakeholder should be pooled

# Risk Management Paradigm

# Risk Identification

- *Product size*—risks associated with the overall size of the software to be built or modified.
- *Business impact*—risks associated with constraints imposed by management or the marketplace.
- *Customer characteristics*—risks associated with the sophistication of the customer and the developer's ability to communicate with the customer in a timely manner.
- *Process definition*—risks associated with the degree to which the software process has been defined and is followed by the development organization.
- *Development environment*—risks associated with the availability and quality of the tools to be used to build the product.
- *Technology to be built*—risks associated with the complexity of the system to be built and the "newness" of the technology that is packaged by the system.
- *Staff size and experience*—risks associated with the overall technical and project experience of the software engineers who will do the work.

# Assessing Project Risk-I

- Have top software and customer managers formally committed to support the project?
- Are end-users enthusiastically committed to the project and the system/product to be built?
- Are requirements fully understood by the software engineering team and their customers?
- Have customers been involved fully in the definition of requirements?
- Do end-users have realistic expectations?

# Assessing Project Risk-II

- Is project scope stable?
- Does the software engineering team have the right mix of skills?
- Are project requirements stable?
- Does the project team have experience with the technology to be implemented?
- Is the number of people on the project team adequate to do the job?
- Do all customer/user constituencies agree on the importance of the project and on the requirements for the system/product to be built?

# Risk Components

- *Performance risk*—the degree of uncertainty that the product will meet its requirements and be fit for its intended use.
- *Cost risk*—the degree of uncertainty that the project budget will be maintained.
- *Support risk*—the degree of uncertainty that the resultant software will be easy to correct, adapt, and enhance.
- *Schedule risk*—the degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time.

# Risk Projection

- *Risk projection*, also called *risk estimation,* attempts to rate each risk in two ways
  - the likelihood or probability that the risk is real
  - the consequences of the problems associated with the risk, should it occur.
- The are four risk projection steps:
  - establish a scale that reflects the perceived likelihood of a risk
  - delineate the consequences of the risk
  - estimate the impact of the risk on the project and the product,
  - note the overall accuracy of the risk projection so that there will be no misunderstandings.

# Building a Risk Table

| Risk | Probability | Impact | RMMM |
|---|---|---|---|
| | | | Risk Mitigation Monitoring & Management |

# Building the Risk Table

- Estimate the probability of occurrence
- Estimate the impact on the project on a scale of 1 to 5, where
  - 1 = low impact on project success
  - 5 = catastrophic impact on project success
- sort the table by probability and impact

# Risk Exposure (Impact)

The overall *risk exposure*, RE, is determined using the following relationship [Hal98]:

$$\mathbf{RE} = P \times C$$

where

$P$ is the probability of occurrence for a risk, and

$C$ is the cost to the project should the risk occur.

# Risk Exposure Example

- **Risk identification.** Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.
- **Risk probability.** 80% (likely).
- **Risk impact.** 60 reusable software components were planned. If only 70 percent can be used, 18 components would have to be developed from scratch (in addition to other custom software that has been scheduled for development). Since the average component is 100 LOC and local data indicate that the software engineering cost for each LOC is $14.00, the overall cost (impact) to develop the components would be 18 x 100 x 14 = $25,200.
- **Risk exposure.** RE = 0.80 x 25,200 ~ $20,200.

# Risk Mitigation, Monitoring, and Management

- Mitigation—how can we avoid the risk?
- Monitoring—what factors can we track that will enable us to determine if the risk is becoming more or less likely?
- Management—what contingency plans do we have if the risk becomes a reality?

# Risk Due to Product Size

*Attributes that affect risk:*

- estimated size of the product in LOC or FP?
- estimated size of product in number of programs, files, transactions?
- percentage deviation in size of product from average for previous products?
- size of database created or used by the product?
- number of users of the product?
- number of projected changes to the requirements for the product? before delivery? after delivery?
- amount of reused software?

# Risk Due to Business Impact

*Attributes that affect risk:*

- affect of this product on company revenue?
- visibility of this product by senior management?
- reasonableness of delivery deadline?
- number of customers who will use this product
- interoperability constraints
- sophistication of end users?
- amount and quality of product documentation that must be produced and delivered to the customer?
- governmental constraints
- costs associated with late delivery?
- costs associated with a defective product?

# Risks Due to the Customer

*Questions that must be answered:*

- Have you worked with the customer in the past?
- Does the customer have a solid idea of requirements?
- Has the customer agreed to spend time with you?
- Is the customer willing to participate in reviews?
- Is the customer technically sophisticated?
- Is the customer willing to let your people do their job—that is, will the customer resist looking over your shoulder during technically detailed work?
- Does the customer understand the software engineering process?

# Risks Due to Process Maturity

*Questions that must be answered:*

- Have you established a common process framework?
- Is it followed by project teams?
- Do you have management support for software engineering
- Do you have a proactive approach to SQA?
- Do you conduct formal technical reviews?
- Are CASE tools used for analysis, design and testing?
- Are the tools integrated with one another?
- Have document formats been established?

# Technology Risks

## *Questions that must be answered:*

- Is the technology new to your organization?
- Are new algorithms, I/O technology required?
- Is new or unproven hardware involved?
- Does the application interface with new software?
- Is a specialized user interface required?
- Is the application radically different?
- Are you using new software engineering methods?
- Are you using unconventional software development methods, such as formal methods, AI-based approaches, artificial neural networks?
- Are there significant performance constraints?
- Is there doubt the functionality requested is "do-able?"

# Staff/People Risks

*Questions that must be answered:*

- Are the best people available?
- Does staff have the right skills?
- Are enough people available?
- Are staff committed for entire duration?
- Will some people work part time?
- Do staff have the right expectations?
- Have staff received necessary training?
- Will turnover among staff be low?

# Recording Risk Information

**Project:** Embedded software for XYZ system
**Risk type:** schedule risk
**Priority (1 low ... 5 critical):** 4
**Risk factor:** Project completion will depend on tests which require hardware component under development. Hardware component delivery may be delayed
**Probability:** 60 %
**Impact:** Project completion will be delayed for each day that hardware is unavailable for use in software testing
**Monitoring approach:**
Scheduled milestone reviews with hardware group
**Contingency plan:**
Modification of testing strategy to accommodate delay using software simulation
**Estimated resources:** 6 additional person months beginning in July

# THANK YOU